java项目中关于权限校验的方式有下面几种

自写鉴权方法

自定义鉴权方法，搭配拦截器或者过滤器进行全局校验，或者在固定方法执行前进行鉴权校验

这种是最容易出现鉴权问题的，大概问题分为下面两种

1、鉴权身份标识问题(存在可伪造性)

这个实战中除了JWT认证，几乎不太能出现了，比如说登录用户成功后，在cookie中增加个可伪造的身份凭据，比如这样cookie：user=admin 或者 cookie：user_id=1

网上也确实有这种案例，但是到今天2025年了，这种实战中出现的概率可以说是几乎没有

要么就是身份标识生成逻辑加了了随机数，存放到数据库中，进行对比，无法伪造

要么就是身份标识伪造需要账号密码。总之就是一大堆情况，所以细讲这个的意义不大了已经

2、鉴权逻辑问题(存在鉴权绕过)

这个有多种情况

1、认证逻辑缺失

这个也不太容易出现，但是我确实是在实际源码审计中有遇到过，简单说一下

参考案例：Echo 4.2审计

https://mp.weixin.qq.com/s/m2EcYyOMig9T_Yd3HP-syw

2、白名单绕过

这个得搭配tomcat

我们来简单写个漏洞demo，就在我们上节课写的servlet_demo的基础上编写吧

创建个用户登录接口

```
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
import java.io.IOException;
import java.io.PrintWriter;

@WebServlet("/Login")
public class Login_Demo extends HttpServlet {

    // 模拟用户数据库
    private static final String VALID_USERNAME = "caigo";
    private static final String VALID_PASSWORD = "123456";

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException {
```

```
        // 设置请求和响应编码
        request.setCharacterEncoding("UTF-8");
        response.setContentType("text/html;charset=UTF-8");

        // 获取表单提交的用户名和密码
        String username = request.getParameter("username");
        String password = request.getParameter("password");

        PrintWriter out = response.getWriter();

        // 验证用户名和密码
        if (VALID_USERNAME.equals(username) && VALID_PASSWORD.equals(password)) {
            // 登录成功

            // 1. 创建Session并设置用户信息
            HttpSession session = request.getSession();
            session.setAttribute("username", username);
            session.setAttribute("loginTime", System.currentTimeMillis());

            // 2. 设置Session过期时间(可选，单位:秒)
            session.setMaxInactiveInterval(30 * 60); // 30分钟

            // 3. 返回登录成功信息
            out.println("<html><body>");
            out.println("<h2>登录成功!</h2>");
            out.println("<p>用户名: " + username + "</p>");
            out.println("<p><a href='UserInfo'>查看用户信息</a></p>");
            out.println("</body></html>");
        } else {
            // 登录失败
            out.println("<html><body>");
            out.println("<h2 style='color:red'>登录失败! 用户名或密码错误</h2>");
            out.println("<p><a href='login.html'>返回登录页面</a></p>");
            out.println("</body></html>");
        }
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException {
        // 如果是GET请求，重定向到登录页面或返回错误
        response.sendRedirect("login.html");
    }
}
```

前端

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>user login</title>
  <style>
```

```
    body { font-family: Arial, sans-serif; max-width: 400px; margin: 0 auto;
padding: 20px; }
    form { border: 1px solid #ccc; padding: 20px; border-radius: 5px; }
    input { width: 100%; padding: 8px; margin: 8px 0; box-sizing: border-box; }
    button { background-color: #4CAF50; color: white; padding: 10px 15px; border:
none; cursor: pointer; }
    button:hover { background-color: #45a049; }
  </style>
</head>
<body>
<h2>user login</h2>
<form action="Login" method="post">
  <label for="username">username:</label>
  <input type="text" id="username" name="username" required>

  <label for="password">password:</label>
  <input type="password" id="password" name="password" required>

  <button type="submit">login</button>
</form>
</body>
</html>
```

然后创建个过滤器

```
package filter;

import javax.servlet.*;
import javax.servlet.annotation.WebFilter;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpSession;
import java.io.IOException;

@WebFilter("/admin")
public class Check_Admin implements Filter {
    @Override
//中间件启动后就自动运行
    public void init(FilterConfig filterConfig) throws ServletException {
        System.out.println("--创建过滤器--");
    }

    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
FilterChain chain) throws IOException, ServletException {
        System.out.println("--过滤器开始工作--");
        HttpServletRequest requests = (HttpServletRequest) request;
        String requestURI = requests.getRequestURI();
        HttpSession session = requests.getSession();
        System.out.println("拦截请求: " + requestURI);
        String username = (String) session.getAttribute("username");
        if (requestURI.contains("login.jsp")){
            chain.doFilter(request, response);

        }else{
            if ("caigo".equals(username)) {
```

```
                chain.doFilter(request, response);
            } else {
                System.out.println("--非管理员--");
                request.getRequestDispatcher("/login.jsp").forward(request,
response);
            }
        }


    }
}
```

这代码都比较符合我们的正常业务对吧，但是这里有个问题

我们在进行url匹配的时候采用的是contains方法，这个方法是模糊匹配，只要我们传入的内容包含定义的内容就判断未true

这里就要撤出我们tomcat的一个路由解析特性了

Tomcat 路由解析中的分号(;)特性

Tomcat 在处理 URL 时有一个特殊行为：它会将分号(;)后面的内容视为路径参数(path parameters)，而不是路由的一部分。这个特性原本是为了支持 JSR-315 (Servlet 3.0) 规范，但可能被利用来绕过安全控制。

小知识：这个特性在J2EE规范中有定义，不仅Tomcat，其他Servlet容器(如Jetty)也有类似行为

什么意思呢，简单来说，当我们访问路由为/index;caigo的时候

在filter中接收到的路由请求是/index;caigo，而到了servlet的时候，会被解析成/index，而caigo会被当成参数

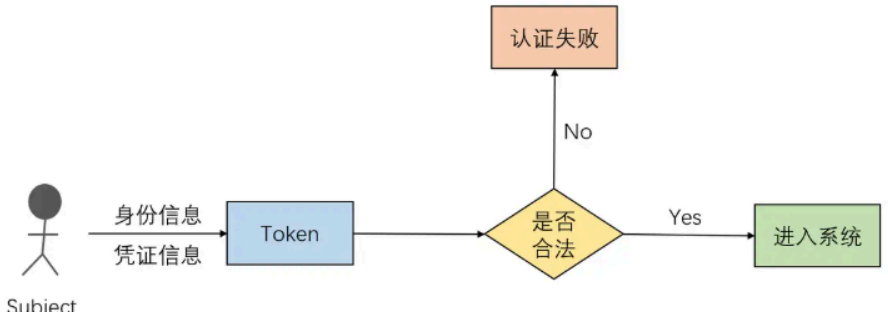
那么带入到我们上面的这个过滤情况，我们是不是可以构造这样的一个路由来绕过filter呢

/admin;login.jsp

使用Apache Shiro

Shiro认证流程

Shiro认证流程



一、导入依赖

pom.xml

```
<dependencies>
    <!-- Shiro Core -->
    <dependency>
        <groupId>org.apache.shiro</groupId>
        <artifactId>shiro-core</artifactId>
        <version>1.9.0</version>
    </dependency>

    <!-- 日志支持（Shiro 使用 SLF4J） -->
    <dependency>
        <groupId>org.slf4j</groupId>
        <artifactId>slf4j-simple</artifactId>
        <version>1.7.36</version>
    </dependency>
</dependencies>
```

二、创建配置文件

在 `resources` 目录中创建 `shiro.ini` 文件

```
[users]
john=123
peter=12345
```

ini是Shiro中的配置文件，在没有连接数据库时可以将系统中相关的权限数据写在ini文件中

我们简单写个小demo就不连接数据库了，原理上是一样的

三、Demo

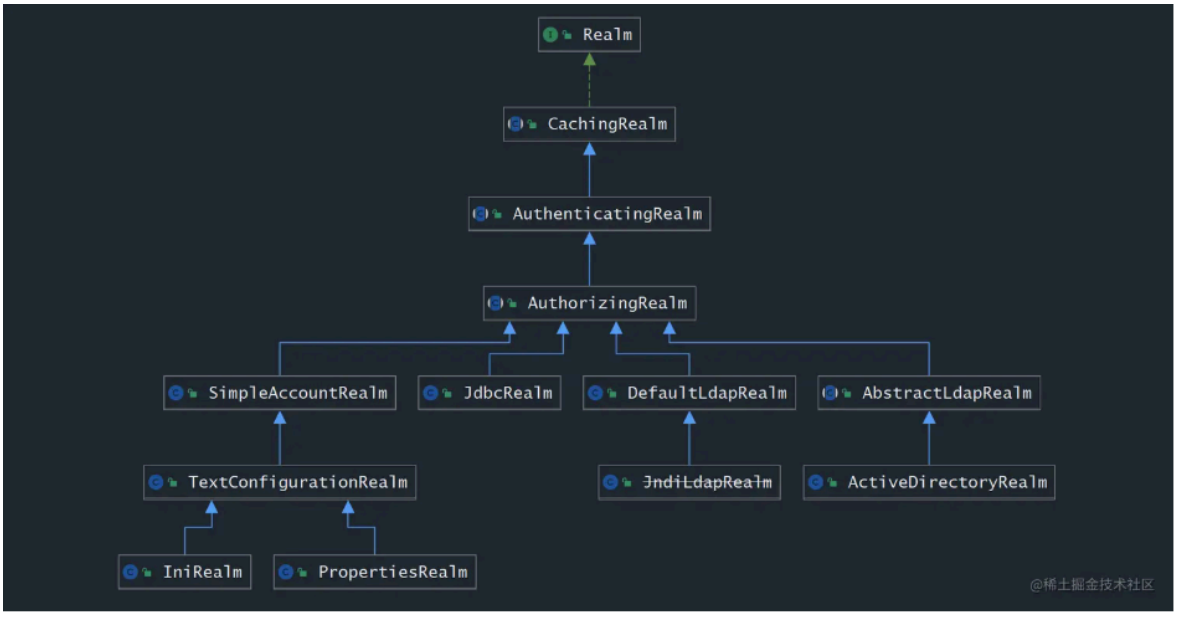
```
public static void main(String[] args) {
    // 创建安全管理器对象 DefaultSecurityManager是对SecurityManager接口的一个具体实现
    DefaultSecurityManager securityManager = new DefaultSecurityManager();

    // 给安全管理器设置Realm 获取身份和凭证数据
    securityManager.setRealm(new IniRealm("classpath:shiro.ini"));

    // Shiro提供的全局安全工具类SecurityUtils 安全工具设置安全管理器完成认证
    SecurityUtils.setSecurityManager(securityManager);

    // 关键对象 Subject 当前登录的主体对象
    Subject subject = SecurityUtils.getSubject();

    // 创建令牌 身份信息为username 凭证信息为password
    UsernamePasswordToken token = new UsernamePasswordToken("john", "123");

    try {
        // 用户认证 认证通过不会抛出异常
        subject.login(token);
        // 获取认证状态 true为认证成功 false为认证失败
        System.out.println("认证状态: " + subject.isAuthenticated());
    } catch (UnknownAccountException e) {
        // 认证失败异常 用户名不存在
        e.printStackTrace();
```

```
    } catch (IncorrectCredentialsException e) {
        // 认证失败异常 密码错误
        e.printStackTrace();
    }
}
```

这个示例展示了 Shiro 认证的基本流程：

1. 配置安全管理器
2. 设置 Realm 数据源
3. 获取当前用户主体
4. 创建认证令牌
5. 执行登录认证
6. 处理认证结果

在实际应用中，通常会使用更复杂的 Realm 实现，如连接数据库的 JDBC Realm，Shiro提供的系统Realm如下：



四、自定义认证

如果我们要将读取数据的模式从ini配置文件改变到从数据库中读取数据，那么我们必须自己定义一个类去继承 `AuthorizingRealm`，并重写 `doGetAuthenticationInfo` 方法。

4.1创建自定义 Realm

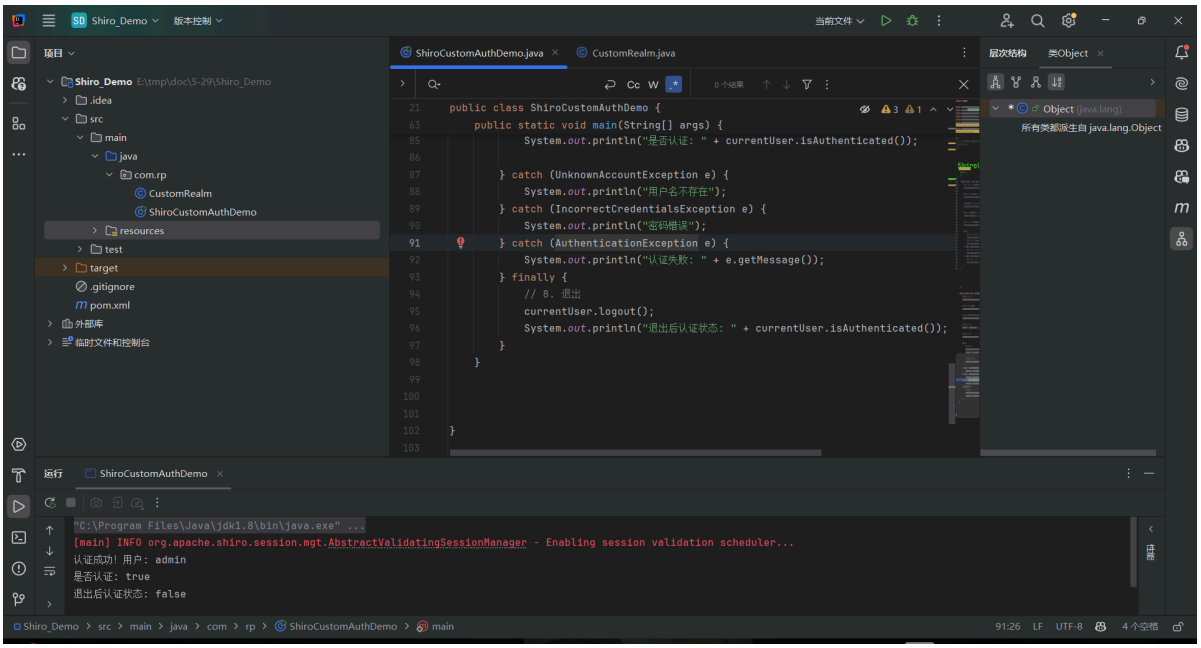
首先创建一个继承自 `AuthorizingRealm` 的类（同时支持认证和授权）：

```
import org.apache.shiro.authc.*;
import org.apache.shiro.authz.AuthorizationInfo;
import org.apache.shiro.realm.AuthorizingRealm;
import org.apache.shiro.subject.PrincipalCollection;

public class CustomRealm extends AuthorizingRealm {
```

```java
    // 设置Realm名称
    public CustomRealm() {
        this.setName("customRealm");
    }

    /**
     * 认证方法
     * @param token 包含用户提交的身份和凭证
     * @return 认证信息
     * @throws AuthenticationException 认证异常
     */
    @Override
    protected AuthenticationInfo doGetAuthenticationInfo(AuthenticationToken
token)
        throws AuthenticationException {

        // 1. 将token转换为UsernamePasswordToken
        UsernamePasswordToken upToken = (UsernamePasswordToken) token;

        // 2. 获取用户名
        String username = upToken.getUsername();

        // 3. 模拟从数据库或其他数据源中查询用户信息
        // 这里应该替换为实际的用户查询逻辑
        if ("admin".equals(username)) {
            // 参数说明:
            // principal: 认证的主体，可以是用户名或用户对象
            // credentials: 从数据库查询出的正确密码
            // realmName: 当前realm的名称
            return new SimpleAuthenticationInfo(
                username,               // principal
                "admin123",             // credentials (正确密码)
                this.getName()           // realmName
            );
        } else if ("user".equals(username)) {
            return new SimpleAuthenticationInfo(
                username,
                "user123",
                this.getName()
            );
        }

        // 4. 用户不存在时返回null，Shiro会抛出UnknownAccountException
        return null;
    }

    /**
     * 授权方法（本例暂不实现）
     */
    @Override
    protected AuthorizationInfo doGetAuthorizationInfo(PrincipalCollection
principals) {
        return null;
    }
}
```

4.2 使用自定义 Realm 进行认证

```
import org.apache.shiro.SecurityUtils;
import org.apache.shiro.authc.*;
import org.apache.shiro.mgt.DefaultSecurityManager;
import org.apache.shiro.subject.Subject;

public class ShiroCustomAuthDemo {

    public static void main(String[] args) {
        // 1. 创建安全管理器
        DefaultSecurityManager securityManager = new DefaultSecurityManager();

        // 2. 设置自定义Realm
        securityManager.setRealm(new CustomRealm());

        // 3. 将SecurityManager绑定到SecurityUtils
        SecurityUtils.setSecurityManager(securityManager);

        // 4. 获取当前用户
        Subject currentUser = SecurityUtils.getSubject();

        // 5. 创建认证令牌
        UsernamePasswordToken token = new UsernamePasswordToken("admin",
"admin123");

        try {
            // 6. 登录认证
            currentUser.login(token);
            System.out.println("认证成功! 用户: " + currentUser.getPrincipal());

            // 7. 检查认证状态
            System.out.println("是否认证: " + currentUser.isAuthenticated());

        } catch (UnknownAccountException e) {
            System.out.println("用户名不存在");
        } catch (IncorrectCredentialsException e) {
            System.out.println("密码错误");
        } catch (AuthenticationException e) {
            System.out.println("认证失败: " + e.getMessage());
        } finally {
            // 8. 退出
            currentUser.logout();
            System.out.println("退出后认证状态: " + currentUser.isAuthenticated());
        }
    }
}
```



运行即可

关于SpringBoot整合Shiro

SpringBoot3整合shiro还是挺多坑的，这部分参考学员写的博客

https://rp-sec.top/2024/12/08/SpringBoot3%E6%95%B4%E5%90%88Shiro%E5%AD%A6%E4%B9%A0/#%E4%BB%80%E4%B9%88%E6%98%AFShiro

审计关注点

白名单配置导致敏感信息泄露

这个白名单一般不是在项目配置文件就是在拦截器中

shiro的权限参数可以参考下表

Filter Name	功能
anno	不需要授权、登录就可以访问。eg:/index
authc	需要登录授权才能访问。eg：/用户中心
authcBasic	Basic HTTP身份验证拦截器
logout	退出拦截器。退出成功后，会redirect到设置的/URI
noSessionCreation	不创建会话连接器
perms	授权拦截器:perm['user:create']
port	端口拦截器.eg:port[80]
rest	rest风格拦截器
roles	角色拦截器。eg：role[administrator]
ssl	ssl拦截器。通过https协议才能通过
user	用户拦截器。eg：登录后（authc），第二次没登陆但是有记住我(remmbner)都可以访问。

除白名单就是shiro的版本漏洞，鉴权方面绕过的

```
    "1.1.0": "CVE-2010-3863 ——(权限绕过)",
    "1.2.3": "CVE-2014-0074/Shiro-460 ——(未授权)",
    "1.3.2": "CVE-2016-6802 ——(权限绕过)",
    "1.5.2": "CVE-2020-1957/Shiro-682 ——(权限绕过)",
    "1.5.3": "CVE-2020-11989/Shiro-782 ——(权限绕过)",
    "1.6.0": "CVE-2020-13933 ——(权限绕过)",
    "1.7.0": "CVE-2020-17510 ——(权限绕过)",
    "1.7.1": "CVE-2020-17523 ——(权限绕过)",
    "1.8.0": "CVE-2021-41303/Shiro-825 ——(权限绕过)",
    "1.9.1": "CVE-2022-32532 ——(权限绕过)",
    "1.10.0": "CVE-2022-40664 ——(权限绕过)",
    "1.11.0": "CVE-2023-22602 ——(权限绕过)"
```

使用Spring Security

一、啥玩意是SpringSecurity

Spring Security 是针对 Spring 项目的安全框架，也是 SpringBoot 底层安全模块默认的技术选型，他可以实现强大的Web安全控制，对于安全控制，我们仅需要引入spring-boot-starter-security 模块，进行少量的配置，即可实现强大的安全管理！

Spring Security的两个主要目标是 “认证” 和 “授权”（访问控制）。

“认证”（Authentication）

身份验证是关于验证您的凭据，如用户名/用户ID和密码，以验证您的身份。

身份验证通常通过用户名和密码完成，有时与身份验证因素结合使用。

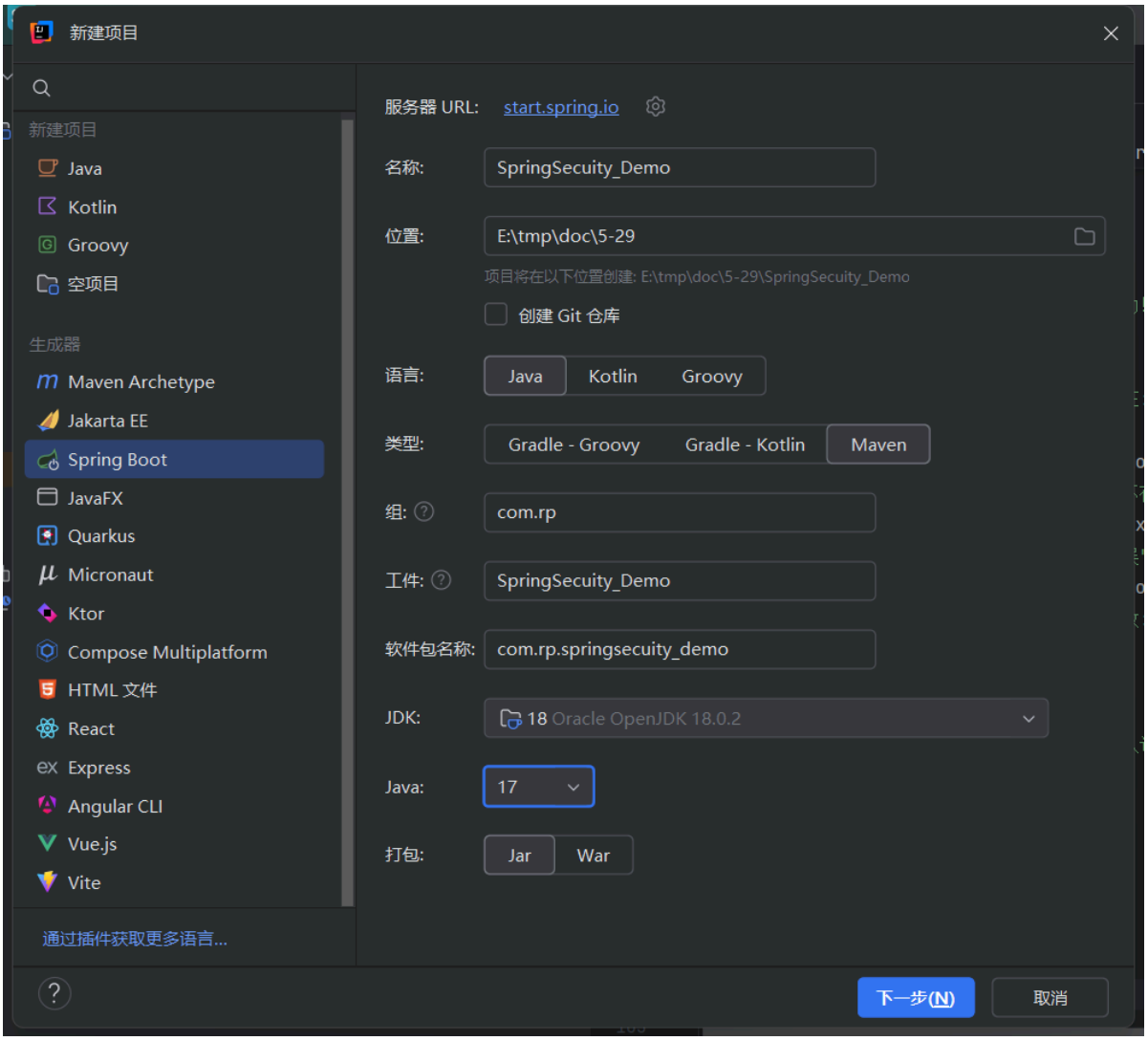
“授权” （Authorization）

授权发生在系统成功验证您的身份后，最终会授予您访问资源（如信息，文件，数据库，资金，位置，几乎任何内容）的完全权限。

这个概念是通用的，而不是只在 Spring Security 中存在。

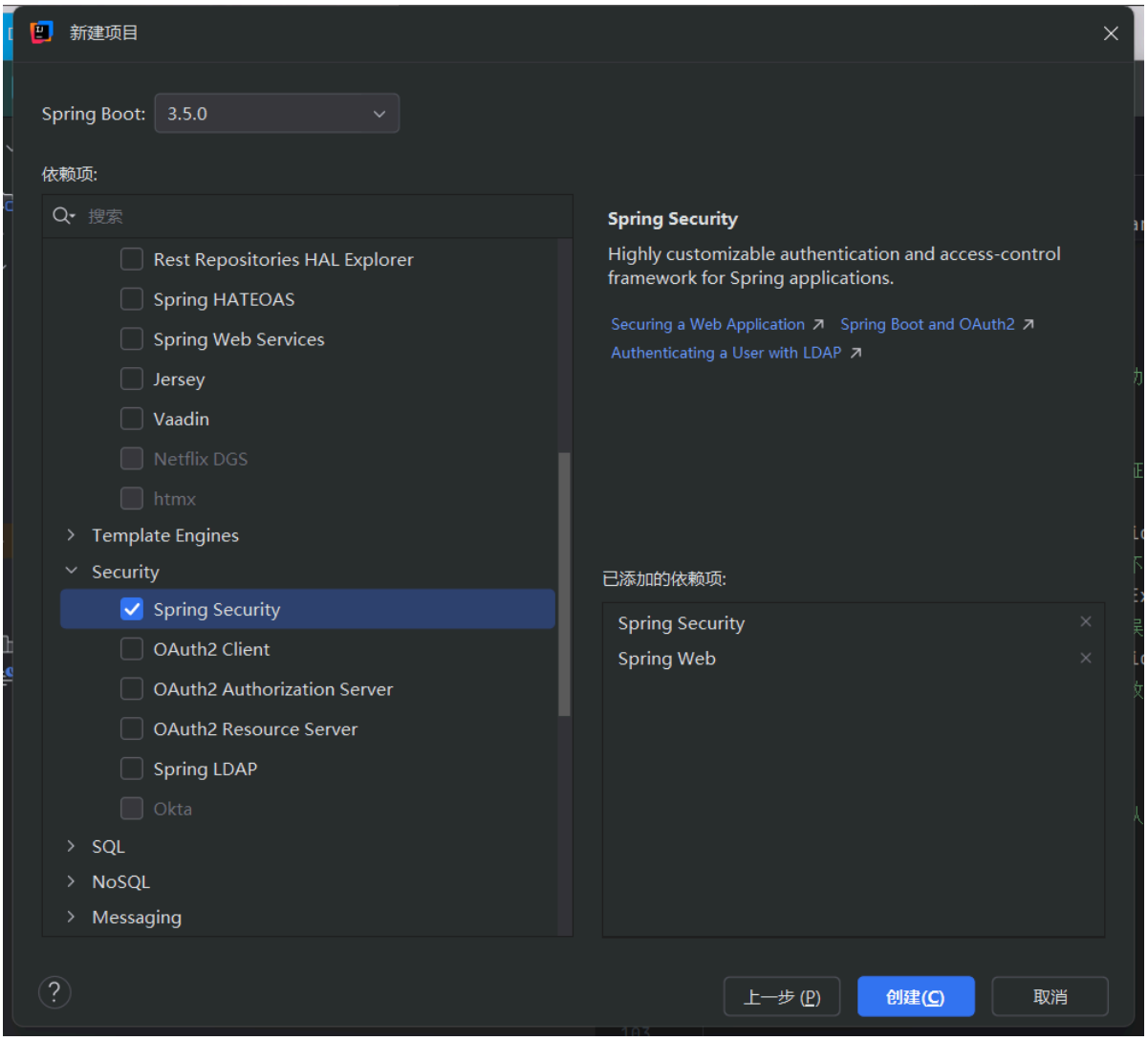
环境搭建

新建项目选择SpringBoot项目



注意SpringBoot3后不支持jdk1.8，这里我用的jdk18

然后安全框架选择Spring Security



Demo编写

现在假设我们有一个网站，有如下前端界面

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>登录页面</title>
</head>
<body>
    <h1>用户登录</h1>

    <!-- 如果有错误信息 -->
    <div th:if="${param.error}">
        <p style="color: red;">用户名或密码错误</p>
    </div>

    <!-- 登出成功提示 -->
    <div th:if="${param.logout}">
        <p style="color: green;">您已成功登出</p>
    </div>

    <form action="/doLogin" method="post">
        <div>
```

```
            <label>用户名: <input type="text" name="username"/></label>
        </div>
        <div>
            <label>密码: <input type="password" name="password"/></label>
        </div>
        <div>
            <button type="submit">登录</button>
        </div>
    </form>
</body>
</html>
```

home.html

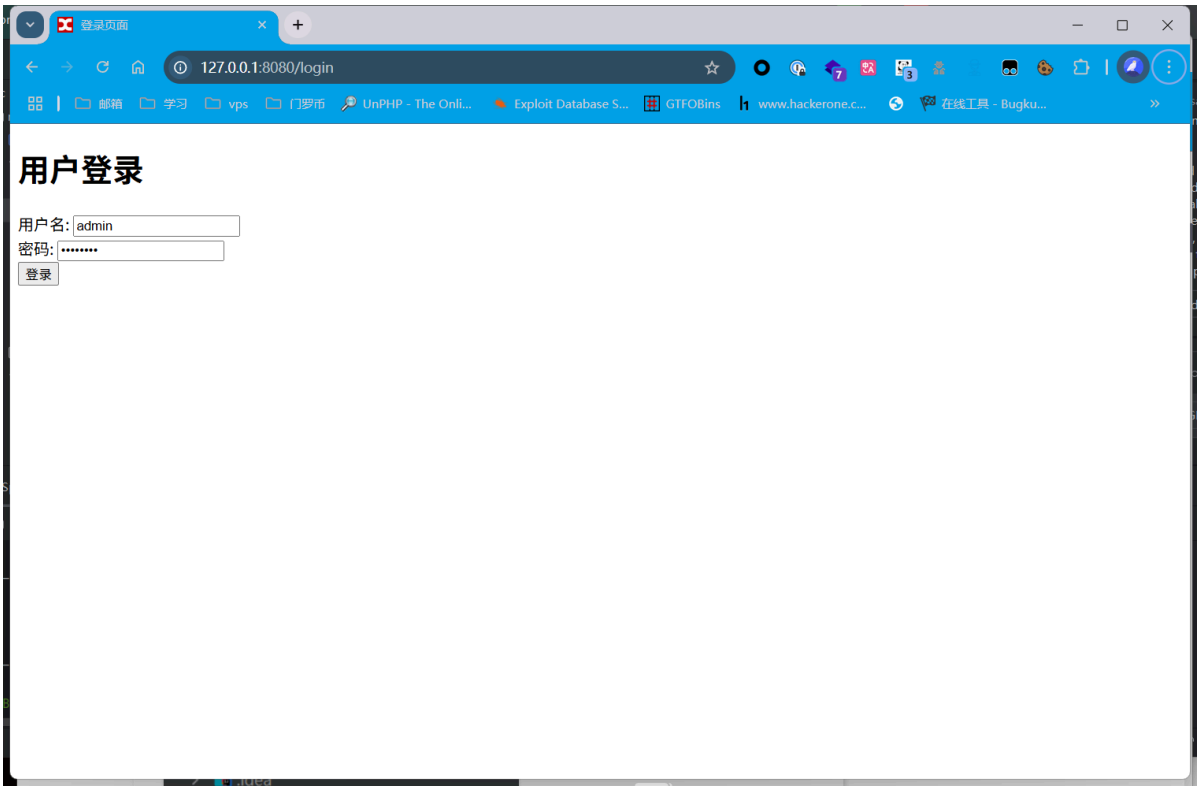
```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>主页</title>
</head>
<body>
    <h1>欢迎来到主页</h1>
    <p>您已成功登录</p>

    <!-- 显示当前用户信息 -->
    <div sec:authorize="isAuthenticated()">
        <p>用户名: <span sec:authentication="name"></span></p>
        <p>角色: <span sec:authentication="principal.authorities"></span></p>
    </div>

    <form th:action="@{/logout}" method="post">
        <button type="submit">退出登录</button>
    </form>
</body>
</html>
```

我们要用SpringSecurity实现拦截鉴权，我们首先要编写SecurityConfiguration

```
package com.study.springbootstudy.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfiguration {
```

```java
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
                .authorizeHttpRequests(auth -> {
                    auth.requestMatchers("/login", "/doLogin").permitAll(); // 允许访问登录页面和登录处理URL
                    auth.anyRequest().authenticated(); // 其他所有请求需要认证
                })
                .formLogin(conf -> {
                    conf.loginPage("/login"); // 自定义登录页面路径
                    conf.loginProcessingUrl("/doLogin"); // 登录处理URL
                    conf.defaultSuccessUrl("/", true); // 登录成功后跳转的URL
                    conf.permitAll(); // 允许所有用户访问登录相关页面
                })
                .logout(logout -> logout
                    .logoutUrl("/logout") // 登出URL
                    .logoutSuccessUrl("/login?logout") // 登出成功后跳转的URL
                    .permitAll()
                )
                .build();
    }

    // 配置内存用户
    @Bean
    public UserDetailsService userDetailsService() {
        UserDetails user = User.builder()
                .username("user")
                .password(passwordEncoder().encode("123456")) // 密码必须加密
                .roles("USER") // 角色
                .build();

        UserDetails admin = User.builder()
                .username("admin")
                .password(passwordEncoder().encode("admin123"))
                .roles("ADMIN", "USER") // 可以拥有多个角色
                .build();

        return new InMemoryUserDetailsManager(user, admin);
    }

    // 密码编码器
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

就可实现鉴权由于篇幅原因代码不全展示出来，如有需要可直接下载源码查看



这个我们审计的时候主要关注它的白名单规则，也是就是无需登录可以访问的接口

主要配置在**SecurityConfig类（最常见）**

```
@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests()
                .antMatchers("/", "/home", "/login", "/public/**").permitAll() //
白名单
                .antMatchers("/admin/**").hasRole("ADMIN") // 权限控制
                .anyRequest().authenticated() // 其他请求需要认证
            .and()
            .formLogin()
                .loginPage("/login")
                .permitAll()
            .and()
            .logout()
                .permitAll();
    }
}
```

5.7+采用SecurityFilterChain

```
@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
```

```
        http
            .authorizeHttpRequests(authorize -> authorize
                .requestMatchers("/", "/home", "/public/**").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .formLogin(form -> form
                .loginPage("/login")
                .permitAll()
            )
            .logout(logout -> logout
                .permitAll()
            );
        return http.build();
    }
}
```

代码审计中的关键检查点

- 必要的公开路径（如`/login`、`/register`）必须明确配置`permitAll()`。
- 静态资源应通过`web.ignoring()`或`permitAll()`放行：

```
// 方式1：完全忽略（不经过Security过滤器链）
web.ignoring().antMatchers("/css/**", "/js/**");

// 方式2：通过permitAll()放行（经过过滤器链但无需认证）
http.authorizeRequests().antMatchers("/public/**").permitAll();
```

JWT认证

什么是JWT?

JWT（JSON Web Token）是一种开放标准（RFC 7519），用于在网络应用环境间安全地传递声明（claims）。它是一种紧凑的、URL安全的表示方式，用于在双方之间传输信息。

主要特点：

- **紧凑**：体积小，可通过URL、POST参数或HTTP头发送
- **自包含**：包含所有必要信息，减少数据库查询
- **可验证**：使用数字签名保证完整性
- **标准化**：遵循RFC 7519标准

代码审计中的注意点

JWT通常是搭配自写鉴权和Spring Security使用的，它只是一种认证方式，鉴权框架没啥关系，除了java，像php，go，python很多项目也会采用

它的安全问题主要有两点

1、加密密钥硬编码

2、加密密钥采用伪随机函数生成

我们一个个来说